

2026-05-05-p1-f08-plan-mode

P1-F08 — Plan Mode Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Add claude-code-style Plan Mode: a read-only operational mode where destructive tools are blocked unless authorised by an approved plan action. Agent enters via `EnterPlanMode` tool; user enters via `/plan` slash command. `ExitPlanMode` and `/plan reject` return to normal. Gate uses an allow-list of always-safe tools and a per-tool key-arg map for matching approved-but-unexecuted plan actions.

Architecture: Extend existing `internal/workflow/planmode/` (which already has `Mode/ModeController/Plan/Planner`) with a new `gating.go` holding `ToolGate`. Add `EnterPlanModeTool` + `ExitPlanModeTool` agent tools. Add `/plan` slash command (show / approve [] / reject / status). Wire `ToolRegistry` with `SetPlanModeGate(*ToolGate)` consulted in both foreground and background dispatch paths.

Tech Stack: Go 1.26, testify v1.11, github.com/spf13/cobra v1.8 (already in go.mod), gopkg.in/yaml.v3 (already in go.mod), go.uber.org/zap (already in go.mod since F07-T03). **No new external dependencies.** Standard-library context, errors, fmt, sync, text/tabwriter, time.

Spec: docs/superpowers/specs/2026-05-05-p1-f08-plan-mode-design.md (commit c7c4843)

Working directory for all go commands: `helix_code/` (the inner Go module). Git commands run from the meta-repo root `/run/media/milosvasic/DATA4TB/Projects/helix_code/` per the F01–F07 convention.

Anti-bluff smoke (run on every commit, FULL pattern):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/workflow/planmode/gating.go \
    internal/tools/plan_tools.go internal/tools/types_planmode.go \
    internal/commands/plan_command.go && echo "BLUFF FOUND" ||
    echo "clean"
```

Task list



P1-F08-T01 — bootstrap evidence + advance PROGRESS



P1-F08-T02 — Planner extensions: `ApprovePlan` / `ApproveAction` / `RejectPlan` / `ActivePlan` + `OnPlanApproval`/`OnPlanReject` hook types (TDD)



P1-F08-T03 — planmode/gating.go: ToolGate with allow-list + key-arg map (TDD)



P1-F08-T04 — tools/types_planmode.go + plan_tools.go: ErrPlanModeGated + EnterPlanModeTool + ExitPlanModeTool (TDD)



P1-F08-T05 — tools/registry.go: SetPlanModeGate + Execute + executeInBackground gating (TDD)



P1-F08-T06 — commands/plan_command.go: /plan slash + builtin registration (TDD)



P1-F08-T07 — cmd/cli/main.go startup wiring + integration test (real Bash gate)



P1-F08-T08 — Challenge with runtime evidence + cross-compile check



P1-F08-T09 — Feature 8 close-out + push to 4 remotes

Task 1: Bootstrap evidence + advance PROGRESS

Files: - Modify: docs/improvements/06_phase_1_evidence.md - Modify: docs/improvements/PROGRESS.md



Step 1: Append F08 section header to evidence file

Append to docs/improvements/06_phase_1_evidence.md:

P1-F08 — Plan Mode

****Spec:**** `docs/superpowers/specs/2026-05-05-p1-f08-plan-mode-design.md` (commit `c7c4843`)

****Plan:**** `docs/superpowers/plans/2026-05-05-p1-f08-plan-mode.md`

****Started:**** 2026-05-05

****Status:**** active

Task evidence trail

(filled in commit-by-commit as tasks land)



Step 2: Update PROGRESS.md current focus

Replace existing “## Current focus” with:

Current focus

- ****Active phase:**** P1 — claude-code feature porting

- ****Active feature:**** F08 – Plan Mode
- ****Active task:**** P1-F08-T01 – bootstrap evidence + advance PROGRESS
- ****Last completed:**** P1-F07-T11 – Feature 7 (Background Task System) close-out + push
- ****Owner:**** agent (Claude Opus 4.7)
- ****Started:**** 2026-05-04
- ****Last touched:**** 2026-05-05
- ****Blocked-on:**** none



Step 3: Add F08 task list block

After the F07 task list block (all 11 items checked), insert:

```
## Active feature task list (P1-F08: Plan Mode)
- [ ] P1-F08-T01 – bootstrap evidence + advance PROGRESS
- [ ] P1-F08-T02 – Planner extensions + hook types (TDD)
- [ ] P1-F08-T03 – planmode/gating.go: ToolGate (TDD)
- [ ] P1-F08-T04 – tools/types_planmode.go + plan_tools.go (TDD)
- [ ] P1-F08-T05 – tools/registry.go: SetPlanModeGate + Execute gating (TDD)
- [ ] P1-F08-T06 – commands/plan_command.go: /plan slash + builtin reg (TDD)
- [ ] P1-F08-T07 – cmd/cli/main.go startup wiring + integration test
- [ ] P1-F08-T08 – Challenge with runtime evidence + cross-compile check
- [ ] P1-F08-T09 – Feature 8 close-out + push
```



Step 4: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
improvements/06_phase_1_evidence.md docs/improvements/
PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
docs(P1-F08-T01): bootstrap Phase 1 / Feature 8 evidence +
advance PROGRESS
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```
EOF
)"
```

Task 2: Planner extensions + hook types (TDD)

Files: - Modify: `helix_code/internal/workflow/planmode/planner.go` — add `ApprovePlan(planID) error`, `ApproveAction(planID, actionID) error`, `RejectPlan(planID) error`, `ActivePlan() *Plan` to the `Planner` interface and its `DefaultPlanner` implementation. Add `PlanApproved / PlanRejected` constants if missing. Fire hooks via the existing `hooks.Manager`. - Modify: `helix_code/internal/hooks/hook.go` — add `OnPlanApproval`, `OnPlanReject` `HookType` constants. - Create: `helix_code/internal/workflow/planmode/planner_approval_test.go` (or extend `planner_test.go` if a test file exists)



Step 1: Read existing `planner.go` + `hook.go`

```
sed -n '1,250p' helix_code/internal/workflow/planmode/planner.go
grep -n "^type HookType\|^const\|HookType =" helix_code/internal/
hooks/hook.go | head -20
```

Identify: - Existing `Planner` interface methods (e.g., `SubmitPlan`, `GetPlan`). - Existing `Plan` struct fields (`ID`, `Actions`, `Status`). - Existing `PlanStatus` constants — check if `PlanApproved / PlanRejected` already exist; if not, add them. - Existing `PlanAction` shape — confirm it has `Approved *bool` and `Executed bool` (or equivalent). - The `hooks.Manager` reference path the planner uses to fire hooks (or whether it uses a separate dispatcher — if `planmode` has its own dispatcher, route through that and skip the `hooks.Manager` direct call).



Step 2: Write failing test

Create `helix_code/internal/workflow/planmode/planner_approval_test.go`:

```
package planmode

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func newPlannerForTest(t *testing.T) Planner {
    t.Helper()
    // Adapt to actual constructor — read planner.go first.
    return NewDefaultPlanner()
}

func samplePlan(id string) *Plan {
    return &Plan{
        ID:      id,
        Title:   "test plan",
    }
}
```

```

        Actions: []PlanAction{
            {ID: "a1", ToolName: "Edit", Args:
map[string]any{"file_path": "foo.go"}},
            {ID: "a2", ToolName: "Bash", Args:
map[string]any{"command": "go test ./..."}},
        },
        Status: PlanPending,
    }
}

func TestPlanner_ApprovePlanMarksAllApproved(t *testing.T) {
    p := newPlannerForTest(t)
    plan := samplePlan("p1")
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.ApprovePlan("p1"))

    got, err := p.GetPlan("p1")
    require.NoError(t, err)
    assert.Equal(t, PlanApproved, got.Status)
    for _, a := range got.Actions {
        require.NotNil(t, a.Approved)
        assert.True(t, *a.Approved, "action %s must be
approved", a.ID)
    }
}

func TestPlanner_ApproveAction_OneOnly(t *testing.T) {
    p := newPlannerForTest(t)
    plan := samplePlan("p2")
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.ApproveAction("p2", "a1"))

    got, _ := p.GetPlan("p2")
    require.NotNil(t, got.Actions[0].Approved)
    assert.True(t, *got.Actions[0].Approved)
    assert.Nil(t, got.Actions[1].Approved)
}

func TestPlanner_RejectPlanSetsStatus(t *testing.T) {
    p := newPlannerForTest(t)
    plan := samplePlan("p3")
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.RejectPlan("p3"))

    got, _ := p.GetPlan("p3")
    assert.Equal(t, PlanRejected, got.Status)
}

```

```

func TestPlanner_ActivePlanReturnsLatestPending(t *testing.T) {
    p := newPlannerForTest(t)
    require.NoError(t, p.SubmitPlan(samplePlan("p1")))
    require.NoError(t, p.SubmitPlan(samplePlan("p2")))

    active := p.ActivePlan()
    require.NotNil(t, active)
    // "Active" = a plan whose status is Pending or Approved
    // (i.e., not Rejected/Completed).
    assert.Contains(t, []string{"p1", "p2"}, active.ID)
}

func TestPlanner_ApproveUnknownPlanErrors(t *testing.T) {
    p := newPlannerForTest(t)
    err := p.ApprovePlan("nonexistent")
    require.Error(t, err)
}

```

If existing `Plan.Status` is an int (`PlanStatus int iota` from existing `planner.go`), `PlanApproved/PlanRejected` are int constants — assertions need `assert.Equal(t, PlanApproved, got.Status)` with proper types. Read `planner.go` and adapt.

If `Planner` is an interface and `DefaultPlanner` doesn't yet expose `SubmitPlan` (the existing API may be different), use whichever method ALREADY adds a plan to storage. The test MUST exercise the actual API.



Step 3: Run failing test

```

cd HelixCode && go test -count=1 -run "TestPlanner_Approve|
    TestPlanner_Reject|TestPlanner_ActivePlan" ./internal/
    workflow/planmode/...

```

Expected: FAIL with undefined `ApprovePlan/RejectPlan/ActivePlan/PlanApproved/PlanRejected`.



Step 4: Add hook types

In `helix_code/internal/hooks/hook.go`, find the existing `HookType` const block and append:

```

// Plan-mode hooks (F08)
OnPlanApproval HookType = "OnPlanApproval"
OnPlanReject    HookType = "OnPlanReject"

```

Verify the block has no trailing comment/comma issues after the addition.



Step 5: Add Planner methods

In `helix_code/internal/workflow/planmode/planner.go`:

1. Add `PlanApproved` and `PlanRejected` to the existing `PlanStatus` enum if not present:

```
const (  
    PlanDraft PlanStatus = iota  
    PlanPending  
    PlanApproved  
    PlanRejected  
    PlanExecuting  
    PlanCompleted  
    PlanFailed  
)
```

Adapt the `String()` method's slice accordingly.

2. Extend the `Planner` interface to include:

```
ApprovePlan(planID string) error  
ApproveAction(planID, actionID string) error  
RejectPlan(planID string) error  
ActivePlan() *Plan
```

3. Implement on `DefaultPlanner`:

```
func (p *DefaultPlanner) ApprovePlan(planID string) error {  
    p.mu.Lock()  
    plan, ok := p.plans[planID]  
    if !ok {  
        p.mu.Unlock()  
        return fmt.Errorf("plan %s not found", planID)  
    }  
    plan.Status = PlanApproved  
    now := time.Now()  
    plan.ApprovedAt = &now  
    for i := range plan.Actions {  
        t := true  
        plan.Actions[i].Approved = &t  
    }  
    hookMgr := p.hooks // captured under lock  
    p.mu.Unlock()  
  
    if hookMgr != nil {  
        _ = hookMgr.Trigger(context.Background(),  
            hooks.Event{  
                Type: hooks.OnPlanApproval,  
                Data: map[string]interface{}{  
                    "plan_id": planID,  
                    "action_count": len(plan.Actions),  
                },  
            })  
    }  
}
```

```

        },
    ))
}
return nil
}

func (p *DefaultPlanner) ApproveAction(planID, actionID
    string) error {
    p.mu.Lock()
    defer p.mu.Unlock()
    plan, ok := p.plans[planID]
    if !ok {
        return fmt.Errorf("plan %s not found", planID)
    }
    for i := range plan.Actions {
        if plan.Actions[i].ID == actionID {
            t := true
            plan.Actions[i].Approved = &t
            return nil
        }
    }
    return fmt.Errorf("action %s not found in plan %s",
        actionID, planID)
}

func (p *DefaultPlanner) RejectPlan(planID string) error {
    p.mu.Lock()
    plan, ok := p.plans[planID]
    if !ok {
        p.mu.Unlock()
        return fmt.Errorf("plan %s not found", planID)
    }
    plan.Status = PlanRejected
    hookMgr := p.hooks
    p.mu.Unlock()

    if hookMgr != nil {
        _ = hookMgr.Trigger(context.Background(),
            hooks.Event{
                Type: hooks.OnPlanReject,
                Data: map[string]interface{}{"plan_id": planID},
            })
    }
    return nil
}

func (p *DefaultPlanner) ActivePlan() *Plan {
    p.mu.RLock()

```



```

defer p.mu.RUnlock()
// Return the most-recently-submitted plan whose status
// is Pending or Approved.
// If multiple, return the one created last.
var active *Plan
for _, plan := range p.plans {
    if plan.Status != PlanPending && plan.Status !=
    PlanApproved {
        continue
    }
    if active == nil ||
    plan.CreatedAt.After(active.CreatedAt) {
        active = plan
    }
}
return active
}

```

ADAPT field names: if `DefaultPlanner.plans` is named differently or stored elsewhere, use the actual storage. If `DefaultPlanner` doesn't have a `hooks` field, add it as an optional dependency:

```

type DefaultPlanner struct {
    // ...existing fields...
    hooks *hooks.Manager // optional; nil disables hook
        firing
}

```

And add a setter if construction doesn't naturally accept it:

```

func (p *DefaultPlanner) SetHooksManager(m *hooks.Manager) {
    p.mu.Lock()
    p.hooks = m
    p.mu.Unlock()
}

```

The import path is `dev.helix.code/internal/hooks` — add it to the `planner.go` imports.



Step 6: Run tests

```

cd HelixCode && go test -count=1 -race -run "TestPlanner_Approve|
TestPlanner_Reject|TestPlanner_ActivePlan" ./internal/
workflow/planmode/...

```

Expected: PASS (5/5).



Step 7: Anti-bluff smoke + full sweep

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/workflow/planmode/planner.go
internal/hooks/hook.go && echo "BLUFF FOUND" || echo
"clean"
cd HelixCode && go test -count=1 -race ./internal/workflow/
planmode/... ./internal/hooks/...
```

Expected: smoke clean (or only pre-existing hits in OTHER files unrelated to this commit). Pre-existing tests in workflow/planmode and hooks must still pass.



Step 8: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
helix_code/internal/workflow/planmode/planner.go
helix_code/internal/workflow/planmode/
planner_approval_test.go helix_code/internal/hooks/
hook.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
feat(P1-F08-T02): Planner ApprovePlan/ApproveAction/RejectPlan/
ActivePlan + OnPlanApproval/OnPlanReject hooks
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 3: planmode/gating.go: ToolGate with allow-list + key-arg map (TDD)

Files: - Create: helix_code/internal/workflow/planmode/gating.go - Create: helix_code/internal/workflow/planmode/gating_test.go



Step 1: Write failing test

Create helix_code/internal/workflow/planmode/gating_test.go:

```
package planmode

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)
```

```

func newGateForTest(t *testing.T) (*ToolGate, ModeController,
    Planner) {
    t.Helper()
    mc := NewModeController()
    p := NewDefaultPlanner()
    return NewToolGate(mc, p), mc, p
}

func TestToolGate_NormalModeNeverBlocks(t *testing.T) {
    g, _, _ := newGateForTest(t)
    blocked, _ := g.IsBlocked("Edit",
        map[string]any{"file_path": "x"})
    assert.False(t, blocked)
}

func TestToolGate_AllowListPasses(t *testing.T) {
    g, mc, _ := newGateForTest(t)
    require.NoError(t, mc.TransitionTo(ModePlan))
    for _, name := range []string{"Read", "Glob", "Grep",
        "View", "TaskOutput"} {
        blocked, _ := g.IsBlocked(name, nil)
        assert.False(t, blocked, "%s should be in allow-list",
            name)
    }
}

func TestToolGate_NoActivePlanBlocks(t *testing.T) {
    g, mc, _ := newGateForTest(t)
    require.NoError(t, mc.TransitionTo(ModePlan))
    blocked, reason := g.IsBlocked("Edit",
        map[string]any{"file_path": "x"})
    assert.True(t, blocked)
    assert.Contains(t, reason, "no active plan")
}

func TestToolGate_ApprovedActionAllows(t *testing.T) {
    g, mc, p := newGateForTest(t)
    require.NoError(t, mc.TransitionTo(ModePlan))
    plan := &Plan{
        ID: "p1",
        Title: "T",
        Actions: []PlanAction{
            {ID: "a1", ToolName: "Edit", Args:
                map[string]any{"file_path": "foo.go"}},
        },
        Status: PlanPending,
    }
    require.NoError(t, p.SubmitPlan(plan))
}

```

```

require.NoError(t, p.ApprovePlan("p1"))

blocked, _ := g.IsBlocked("Edit",
    map[string]any{"file_path": "foo.go"})
assert.False(t, blocked)
}

func TestToolGate_KeyArgMismatchBlocks(t *testing.T) {
    g, mc, p := newGateForTest(t)
    require.NoError(t, mc.TransitionTo(ModePlan))
    plan := &Plan{
        ID: "p1",
        Actions: []PlanAction{
            {ID: "a1", ToolName: "Edit", Args:
                map[string]any{"file_path": "foo.go"}},
        },
        Status: PlanPending,
    }
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.ApprovePlan("p1"))

    blocked, reason := g.IsBlocked("Edit",
        map[string]any{"file_path": "OTHER.go"})
    assert.True(t, blocked)
    assert.Contains(t, reason, "no approved")
}

func TestToolGate_ExecutedActionDoesNotReauthorise(t *testing.T)
{
    g, mc, p := newGateForTest(t)
    require.NoError(t, mc.TransitionTo(ModePlan))
    plan := &Plan{
        ID: "p1",
        Actions: []PlanAction{
            {ID: "a1", ToolName: "Edit", Args:
                map[string]any{"file_path": "foo.go"}},
        },
        Status: PlanPending,
    }
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.ApprovePlan("p1"))

    // Match + mark executed.
    matched, ok := g.MatchApprovedAction("Edit",
        map[string]any{"file_path": "foo.go"})
    require.True(t, ok)
    g.MarkExecuted(plan.ID, matched.ID)
}

```

```

    // Subsequent Edit on same file_path is now blocked.
    blocked, _ := g.IsBlocked("Edit",
        map[string]any{"file_path": "foo.go"})
    assert.True(t, blocked)
}

func TestToolGate_NoKeyArgFallsBackToNameOnly(t *testing.T) {
    g, mc, p := newGateForTest(t)
    require.NoError(t, mc.TransitionTo(ModePlan))
    plan := &Plan{
        ID: "p1",
        Actions: []PlanAction{
            // Hypothetical tool with no key-arg in
            // DefaultKeyArgMap.
            {ID: "a1", ToolName: "CustomTool", Args: nil},
        },
        Status: PlanPending,
    }
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.ApprovePlan("p1"))

    blocked, _ := g.IsBlocked("CustomTool",
        map[string]any{"any": "thing"})
    assert.False(t, blocked)
}

func TestToolGate_WithAllowListExtends(t *testing.T) {
    g, mc, _ := newGateForTest(t)
    g2 := g.WithAllowList([]string{"MyCustomReader"})
    require.NoError(t, mc.TransitionTo(ModePlan))
    blocked, _ := g2.IsBlocked("MyCustomReader", nil)
    assert.False(t, blocked)
    // Original gate still has only defaults
    blocked2, _ := g.IsBlocked("MyCustomReader", nil)
    assert.True(t, blocked2)
}

```



Step 2: Run failing test

```

cd HelixCode && go test -count=1 -run "TestToolGate" ./internal/
workflow/planmode/...

```

Expected: FAIL with undefined ToolGate, NewToolGate, etc.



Step 3: Write gating.go

Create helix_code/internal/workflow/planmode/gating.go:

```

package planmode

import (
    "sync"
)

// DefaultAllowList is the canonical claude-code-style allow-list
// of always-safe
// tools that may run in plan mode without explicit approval.
var DefaultAllowList = []string{
    "Read", "Glob", "Grep", "View", "LSPGetDiagnostics",
    "TaskOutput", "TaskStop", "WebFetch", "WebSearch",
}

// DefaultKeyArgMap maps each destructive tool name to the param
// key whose
// value identifies the action target. An approved plan action
// matches an
// Execute call only when the values for that key are equal
// (where defined).
var DefaultKeyArgMap = map[string]string{
    "Edit":      "file_path",
    "Write":     "file_path",
    "MultiEdit": "file_path",
    "NotebookEdit": "notebook_path",
    "Bash":      "command",
    "shell":     "command",
}

// ToolGate decides whether a tool call is allowed in plan mode.
// A new ToolGate uses DefaultAllowList and DefaultKeyArgMap; use
// WithAllowList to add project-specific safe tools.
type ToolGate struct {
    mc      ModeController
    planner Planner
    allowList map[string]bool
    keyArgs  map[string]string

    mu      sync.Mutex
    executed map[string]map[string]bool // planID → actionID →
                                executed
}

// NewToolGate builds a gate using the default allow-list and
// key-arg map.
func NewToolGate(mc ModeController, planner Planner) *ToolGate {
    g := &ToolGate{
        mc:      mc,

```

```

    planner:    planner,
    allowList:  make(map[string]bool, len(DefaultAllowList)),
    keyArgs:    make(map[string]string,
        len(DefaultKeyArgMap)),
    executed:   make(map[string]map[string]bool),
}
for _, n := range DefaultAllowList {
    g.allowList[n] = true
}
for k, v := range DefaultKeyArgMap {
    g.keyArgs[k] = v
}
return g
}

```

// WithAllowList returns a NEW gate with the given extra tool names added to

// the allow-list. The original gate is unchanged.

```

func (g *ToolGate) WithAllowList(extra []string) *ToolGate {
    c := &ToolGate{
        mc:      g.mc,
        planner:  g.planner,
        allowList: make(map[string]bool, len(g.allowList)
            +len(extra)),
        keyArgs:  make(map[string]string, len(g.keyArgs)),
        executed: make(map[string]map[string]bool),
    }
    for k := range g.allowList {
        c.allowList[k] = true
    }
    for k, v := range g.keyArgs {
        c.keyArgs[k] = v
    }
    for _, n := range extra {
        c.allowList[n] = true
    }
    return c
}

```

// IsBlocked reports whether the named tool call is blocked by plan mode.

// Returns (false, "") in normal mode, on allow-listed tools, or when an

// approved-but-unexecuted plan action authorises the call.

```

func (g *ToolGate) IsBlocked(toolName string, params
    map[string]any) (bool, string) {
    if g.mc.GetMode() != ModePlan {
        return false, ""
    }
}

```

```

    }
    if g.allowList[toolName] {
        return false, ""
    }
    plan := g.planner.ActivePlan()
    if plan == nil {
        return true, "no active plan"
    }
    if _, ok := g.matchActionLocked(plan, toolName, params); ok {
        return false, ""
    }
    return true, "no approved plan action authorises this tool"
}

// MatchApprovedAction returns the first approved-but-unexecuted
// plan action
// authorising the call, or (nil, false).
func (g *ToolGate) MatchApprovedAction(toolName string, params
    map[string]any) (*PlanAction, bool) {
    if g.mc.GetMode() != ModePlan {
        return nil, false
    }
    plan := g.planner.ActivePlan()
    if plan == nil {
        return nil, false
    }
    return g.matchActionLocked(plan, toolName, params)
}

func (g *ToolGate) matchActionLocked(plan *Plan, toolName
    string, params map[string]any) (*PlanAction, bool) {
    keyArg, hasKey := g.keyArgs[toolName]
    g.mu.Lock()
    defer g.mu.Unlock()
    planExec := g.executed[plan.ID]
    for i := range plan.Actions {
        a := &plan.Actions[i]
        if a.ToolName != toolName {
            continue
        }
        if a.Approved == nil || !*a.Approved {
            continue
        }
        if planExec != nil && planExec[a.ID] {
            continue
        }
        if hasKey {
            plannedVal, _ := a.Args[keyArg]

```



```

        actualVal, _ := params[keyArg]
        if !valuesEqual(plannedVal, actualVal) {
            continue
        }
    }
    return a, true
}
return nil, false
}

// MarkExecuted records that a matched action has been consumed.
func (g *ToolGate) MarkExecuted(planID, actionID string) {
    g.mu.Lock()
    defer g.mu.Unlock()
    if g.executed[planID] == nil {
        g.executed[planID] = make(map[string]bool)
    }
    g.executed[planID][actionID] = true
}

// valuesEqual is a strict equality check that handles common
// scalar types.
func valuesEqual(a, b any) bool {
    if a == nil && b == nil {
        return true
    }
    if a == nil || b == nil {
        return false
    }
    switch av := a.(type) {
    case string:
        bv, ok := b.(string)
        return ok && av == bv
    case int:
        bv, ok := b.(int)
        return ok && av == bv
    case float64:
        bv, ok := b.(float64)
        return ok && av == bv
    case bool:
        bv, ok := b.(bool)
        return ok && av == bv
    default:
        return false
    }
}

```

If the existing `Plan.Actions` field is a different type / has different field names, ADAPT. The tests rely on `PlanAction.{ID, ToolName, Approved, Args}` — these MUST be present.



Step 4: Run tests

```
cd HelixCode && go test -count=1 -race -run "TestToolGate" ./
    internal/workflow/planmode/...
```

Expected: PASS (8/8).



Step 5: Anti-bluff smoke + full sweep

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
    placeholder" internal/workflow/planmode/gating.go
    internal/workflow/planmode/gating_test.go && echo "BLUFF
    FOUND" || echo "clean"
cd HelixCode && go test -count=1 -race ./internal/workflow/
    planmode/...
```

Expected: smoke clean. Full sweep clean (T02 + T03 tests + pre-existing planmode tests).



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/workflow/planmode/gating.go
    helix_code/internal/workflow/planmode/gating_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
    feat(P1-F08-T03): add ToolGate with allow-list + key-arg matching
    for plan-mode gating
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```
EOF
)"
```

Task 4: tools/types_planmode.go + plan_tools.go (TDD)

Files: - Create: `helix_code/internal/tools/types_planmode.go` - Create: `helix_code/internal/tools/plan_tools.go` - Create: `helix_code/internal/tools/plan_tools_test.go`



Step 1: Write failing test

Create `helix_code/internal/tools/plan_tools_test.go`:

```

package tools

import (
    "context"
    "errors"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/workflow/planmode"
)

func TestEnterPlanModeTool_TransitionsToPlanMode(t *testing.T) {
    mc := planmode.NewModeController()
    tool := NewEnterPlanModeTool(mc)

    res, err := tool.Execute(context.Background(),
        map[string]interface{}{})
    require.NoError(t, err)
    assert.Equal(t, planmode.ModePlan, mc.GetMode())
    m := res.(map[string]interface{})
    assert.Equal(t, "plan", m["mode"])
}

func TestEnterPlanModeTool_FromInvalidStateErrors(t *testing.T) {
    mc := planmode.NewModeController()
    require.NoError(t, mc.TransitionTo(planmode.ModePlan))
    require.NoError(t, mc.TransitionTo(planmode.ModeAct))
    tool := NewEnterPlanModeTool(mc)

    // Mode Act -> Plan is not a valid transition (per existing
    // transition table)
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{})
    require.Error(t, err)
}

func TestExitPlanModeTool_TransitionsToNormal(t *testing.T) {
    mc := planmode.NewModeController()
    require.NoError(t, mc.TransitionTo(planmode.ModePlan))

    tool := NewExitPlanModeTool(mc)
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{})
    require.NoError(t, err)
    assert.Equal(t, planmode.ModeNormal, mc.GetMode())
}

```

```

func TestErrPlanModeGated_IsExported(t *testing.T) {
    assert.NotNil(t, ErrPlanModeGated)
    wrapped := errors.New("wrap: " + ErrPlanModeGated.Error())
    _ = wrapped
}

func TestEnterPlanModeTool_ImplementsToolInterface(t *testing.T)
{
    var _ Tool = (*EnterPlanModeTool)(nil)
    var _ Tool = (*ExitPlanModeTool)(nil)
}

```



Step 2: Run failing test

```

cd HelixCode && go test -count=1 -run "TestEnterPlanModeTool|
    TestExitPlanModeTool|TestErrPlanModeGated" ./internal/
    tools/

```

Expected: FAIL with undefined.



Step 3: Write types_planmode.go

Create helix_code/internal/tools/types_planmode.go:

```

package tools

import "errors"

// ErrPlanModeGated is returned by ToolRegistry.Execute when a
// destructive
// tool is invoked in plan mode without an approved plan action
// authorising it.
var ErrPlanModeGated = errors.New("tools: blocked by plan mode")

```



Step 4: Write plan_tools.go

Create helix_code/internal/tools/plan_tools.go:

```

package tools

import (
    "context"
    "fmt"

    "dev.helix.code/internal/workflow/planmode"
)

```

```

// EnterPlanModeTool is the agent-callable tool that switches the
    agent
// into plan mode. Side-effect:
    ModeController.TransitionTo(ModePlan).
type EnterPlanModeTool struct {
    mc planmode.ModeController
}

func NewEnterPlanModeTool(mc planmode.ModeController)
    *EnterPlanModeTool {
    return &EnterPlanModeTool{mc: mc}
}

func (t *EnterPlanModeTool) Name() string { return
    "EnterPlanMode" }
func (t *EnterPlanModeTool) Description() string {
    return "Enter plan mode: a read-only operational mode where
        destructive tools are blocked unless authorised by an
        approved plan action."
}
func (t *EnterPlanModeTool) Category() ToolCategory { return
    CategoryFileSystem }
func (t *EnterPlanModeTool) Schema() ToolSchema {
    return ToolSchema{
        Type: "object",
        Properties: map[string]interface{}{},
        Required: []string{},
    }
}
func (t *EnterPlanModeTool) Validate(params
    map[string]interface{}) error { return nil }
func (t *EnterPlanModeTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    if err := t.mc.TransitionTo(planmode.ModePlan); err != nil {
        return nil, fmt.Errorf("enter plan mode: %w", err)
    }
    return map[string]interface{}{
        "mode": "plan",
        "message": "Plan mode active. Propose a plan via the
            planner; user approves via /plan approve.",
    }, nil
}

// ExitPlanModeTool returns the agent to normal mode.
type ExitPlanModeTool struct {
    mc planmode.ModeController
}

```

```

func NewExitPlanModeTool(mc planmode.ModeController)
    *ExitPlanModeTool {
    return &ExitPlanModeTool{mc: mc}
}

func (t *ExitPlanModeTool) Name() string { return
    "ExitPlanMode" }
func (t *ExitPlanModeTool) Description() string {
    return "Exit plan mode and return to normal execution.
    Optional allowed_prompts param (claude-code parity,
    currently informational)."}
}
func (t *ExitPlanModeTool) Category() ToolCategory { return
    CategoryFileSystem }
func (t *ExitPlanModeTool) Schema() ToolSchema {
    return ToolSchema{
        Type: "object",
        Properties: map[string]interface{}{
            "allowed_prompts": map[string]interface{}{
                "type": "array",
                "description": "User prompts that would re-enter
                plan mode (informational; consult F05 hooks for
                enforcement).",
            },
        },
    }
}
func (t *ExitPlanModeTool) Validate(params
    map[string]interface{}) error { return nil }
func (t *ExitPlanModeTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    if err := t.mc.TransitionTo(planmode.ModeNormal); err != nil
    {
        return nil, fmt.Errorf("exit plan mode: %w", err)
    }
    return map[string]interface{}{
        "mode": "normal",
        "message": "Returned to normal execution.",
    }, nil
}

// Compile-time interface assertions.
var (
    _ Tool = (*EnterPlanModeTool)(nil)
    _ Tool = (*ExitPlanModeTool)(nil)
)

```



Step 5: Run tests

```
cd HelixCode && go test -count=1 -race -run
    "TestEnterPlanModeTool|TestExitPlanModeTool|
    TestErrPlanModeGated" ./internal/tools/
```

Expected: PASS (5/5).



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/tools/types_planmode.go internal/
tools/plan_tools.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/tools/types_planmode.go helix_code/
    internal/tools/plan_tools.go helix_code/internal/tools/
    plan_tools_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F08-T04): add EnterPlanMode + ExitPlanMode agent tools +
ErrPlanModeGated sentinel
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```
EOF
)"
```

Task 5: tools/registry.go: SetPlanModeGate + Execute gating (TDD)

Files: - Modify: helix_code/internal/tools/registry.go - Create:
helix_code/internal/tools/registry_planmode_test.go



Step 1: Write failing test

Create helix_code/internal/tools/registry_planmode_test.go:

```
package tools

import (
    "context"
```

```

"errors"
"testing"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"

"dev.helix.code/internal/workflow/planmode"
)

func newRegistryWithGate(t *testing.T) (*ToolRegistry,
    planmode.ModeController, planmode.Planner) {
    t.Helper()
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    mc := planmode.NewModeController()
    p := planmode.NewDefaultPlanner()
    r.SetPlanModeGate(planmode.NewToolGate(mc, p))
    return r, mc, p
}

func TestRegistry_PlanModeGateBlocksDestructive(t *testing.T) {
    r, mc, _ := newRegistryWithGate(t)
    tool := &fakePlainTool{name: "Edit", finalResult: "ok"}
    r.Register(tool)
    require.NoError(t, mc.TransitionTo(planmode.ModePlan))

    _, err := r.Execute(context.Background(), "Edit",
        map[string]interface{}{"file_path": "x.go"})
    require.Error(t, err)
    assert.True(t, errors.Is(err, ErrPlanModeGated))
}

func TestRegistry_PlanModeGateAllowsAllowList(t *testing.T) {
    r, mc, _ := newRegistryWithGate(t)
    tool := &fakePlainTool{name: "Read", finalResult: "content"}
    r.Register(tool)
    require.NoError(t, mc.TransitionTo(planmode.ModePlan))

    res, err := r.Execute(context.Background(), "Read",
        map[string]interface{}{"file_path": "x.go"})
    require.NoError(t, err)
    assert.Equal(t, "content", res)
}

func TestRegistry_PlanModeGateAllowsApprovedAction(t *testing.T)
{
    r, mc, p := newRegistryWithGate(t)
    tool := &fakePlainTool{name: "Edit", finalResult: "edited"}
    r.Register(tool)

```



```

require.NoError(t, mc.TransitionTo(planmode.ModePlan))

plan := &planmode.Plan{
    ID:      "p1",
    Title:   "T",
    Actions: []planmode.PlanAction{
        {ID: "a1", ToolName: "Edit", Args:
            map[string]any{"file_path": "foo.go"}},
    },
    Status: planmode.PlanPending,
}
require.NoError(t, p.SubmitPlan(plan))
require.NoError(t, p.ApprovePlan("p1"))

res, err := r.Execute(context.Background(), "Edit",
    map[string]interface{}{"file_path": "foo.go"})
require.NoError(t, err)
assert.Equal(t, "edited", res)
}

func TestRegistry_PlanModeGateNormalModePassesThrough(t
    *testing.T) {
    r, _, _ := newRegistryWithGate(t)
    tool := &fakePlainTool{name: "Edit", finalResult: "ok"}
    r.Register(tool)
    // mode is Normal – gate is consulted but returns false.

    res, err := r.Execute(context.Background(), "Edit",
        map[string]interface{}{"file_path": "x.go"})
    require.NoError(t, err)
    assert.Equal(t, "ok", res)
}

```



Step 2: Run failing test

```

cd HelixCode && go test -count=1 -run "TestRegistry_PlanMode" ./
    internal/tools/

```

Expected: FAIL with r.SetPlanModeGate undefined.



Step 3: Modify registry.go

Add to imports: "dev.helix.code/internal/workflow/planmode".

Add field to ToolRegistry struct:

```

type ToolRegistry struct {
    // ...existing fields...

```

```

    planGate *planmode.ToolGate
}

```

Add the setter:

```

// SetPlanModeGate wires a plan-mode gate. When set, Execute
// consults it
// before running any tool; blocked calls return
// ErrPlanModeGated.
func (r *ToolRegistry) SetPlanModeGate(g *planmode.ToolGate) {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.planGate = g
}

```

In Execute, ADD the gate check NEAR THE TOP — after the `run_in_background` shortcut (since the `bg` shortcut redirects to `executeInBackground` which we will also patch), BUT BEFORE the existing `fireBefore` + tool resolution. Place it like:

```

func (r *ToolRegistry) Execute(ctx context.Context, name string,
    params map[string]interface{}) (interface{}, error) {
    if bg, ok := params["run_in_background"].(bool); ok && bg {
        return r.executeInBackground(ctx, name, params)
    }
    // F08: plan-mode gate
    if err := r.checkPlanModeGate(name, params); err != nil {
        return nil, err
    }
    // ...existing fireBefore + tool resolution + Execute...
}

```

Add the helper:

```

// checkPlanModeGate consults the plan-mode gate (if wired).
// Returns
// ErrPlanModeGated wrapped with tool name + reason when blocked.
func (r *ToolRegistry) checkPlanModeGate(name string, params
    map[string]interface{}) error {
    r.mu.RLock()
    g := r.planGate
    r.mu.RUnlock()
    if g == nil {
        return nil
    }
    blocked, reason := g.IsBlocked(name, params)
    if !blocked {
        return nil
    }
}

```

```

        return fmt.Errorf("%w: %s (%s)", ErrPlanModeGated, name,
            reason)
    }
}

```

Also call this at the top of `executeInBackground` (after the `bgManager == nil` check, before `r.Get`):

```

func (r *ToolRegistry) executeInBackground(ctx context.Context,
    name string, params map[string]interface{})
    (interface{}, error) {
    // ...existing bgManager + hooksManager capture...
    if err := r.checkPlanModeGate(name, params); err != nil {
        return nil, err
    }
    // ...rest unchanged...
}

```

After successful tool execution in the foreground path, mark the matched plan action as executed. Find the place in `Execute` where the tool's result is returned (after `fireAfter`) and add:

```

// F08: if plan mode authorised this call, mark the action
    executed.
if r.planGate != nil {
    if matched, ok := r.planGate.MatchApprovedAction(name,
        params); ok {
        // Capture plan ID under the planner's lock.
        if plan := getActivePlanID(r.planGate); plan != "" {
            r.planGate.MarkExecuted(plan, matched.ID)
        }
    }
}
}

```

Actually simpler: have `MatchApprovedAction` return the `planID` along with the action. Update the gate (in `gating.go`) to return (`planID string, action *PlanAction, ok bool`):

If the test in `T03` uses the existing `MatchApprovedAction(toolName, params) (*PlanAction, bool)` signature, change it now to return (`planID string, action *PlanAction, ok bool`) so registry can mark executed without a separate query. Update `gating_test.go` accordingly:

```

matched, _, ok := g.MatchApprovedAction("Edit",
    map[string]any{"file_path": "foo.go"})
require.True(t, ok)
g.MarkExecuted(plan.ID, matched.ID)

```

And in `registry.go`:

```

if r.planGate != nil {
    if planID, matched, ok :=
        r.planGate.MatchApprovedAction(name, params); ok {

```

```

        r.planGate.MarkExecuted(planID, matched.ID)
    }
}

```

ADAPT the gating.go signature in T03 if you've already shipped T03 — extend MatchApprovedAction to also return the planID. Update gating_test.go too. (Or keep T03 as-is and have the registry pull planID from g.planner.ActivePlan().ID — works as long as ActivePlan is stable across the call. Either approach is fine; pick whichever results in less touched code.)



Step 4: Run tests

```

cd HelixCode && go test -count=1 -race -run
    "TestRegistry_PlanMode" ./internal/tools/

```

Expected: PASS (4/4).



Step 5: Full tools sweep + smoke

```

cd HelixCode && go test -count=1 -race ./internal/tools/
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/tools/registry.go internal/tools/
    registry_planmode_test.go && echo "BLUFF FOUND" || echo
    "clean"

```

Expected: full sweep passes (existing F02-F07 tests + new F08 tests). Smoke clean.



Step 6: Commit

```

git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/tools/registry.go helix_code/
    internal/tools/registry_planmode_test.go helix_code/
    internal/workflow/planmode/gating.go helix_code/internal/
    workflow/planmode/gating_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F08-T05): ToolRegistry.SetPlanModeGate + Execute gating +
    MarkExecuted on success

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

```

EOF
)"

```

Task 6: commands/plan_command.go: /plan slash + builtin registration (TDD)

Files: - Create: helix_code/internal/commands/plan_command.go - Create: helix_code/internal/commands/plan_command_test.go - Modify: helix_code/internal/commands/builtin/register.go — add RegisterBuiltinCommandsWithPlanMode(registry, planner, mc) - Create: helix_code/internal/commands/builtin/plan_register_test.go



Step 1: Write failing slash test

Create helix_code/internal/commands/plan_command_test.go:

```
package commands

import (
    "context"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/workflow/planmode"
)

func newPlanCommand(t *testing.T) (*PlanCommand,
    planmode.Planner, planmode.ModeController) {
    t.Helper()
    p := planmode.NewDefaultPlanner()
    mc := planmode.NewModeController()
    return NewPlanCommand(p, mc), p, mc
}

func samplePlanForTest() *planmode.Plan {
    return &planmode.Plan{
        ID: "p1",
        Title: "Test Plan",
        Actions: []planmode.PlanAction{
            {ID: "a1", ToolName: "Edit", Args:
                map[string]any{"file_path": "foo.go"}, Description:
                "edit foo"},
            {ID: "a2", ToolName: "Bash", Args:
                map[string]any{"command": "go test ./..."}, Description:
                "run tests"},
        },
        Status: planmode.PlanPending,
    }
}
```

```

func TestSlashPlan_ShowEmptyPlan(t *testing.T) {
    c, _, _ := newPlanCommand(t)
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"show"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "No active plan")
}

func TestSlashPlan_ShowActivePlan(t *testing.T) {
    c, p, _ := newPlanCommand(t)
    require.NoError(t, p.SubmitPlan(samplePlanForTest()))
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"show"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "Test Plan")
    assert.Contains(t, res.Output, "Edit")
    assert.Contains(t, res.Output, "Bash")
}

func TestSlashPlan_ApproveAll(t *testing.T) {
    c, p, _ := newPlanCommand(t)
    require.NoError(t, p.SubmitPlan(samplePlanForTest()))
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"approve"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "approved")

    got, _ := p.GetPlan("p1")
    assert.Equal(t, planmode.PlanApproved, got.Status)
}

func TestSlashPlan_ApproveSingleAction(t *testing.T) {
    c, p, _ := newPlanCommand(t)
    require.NoError(t, p.SubmitPlan(samplePlanForTest()))
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"approve", "a1"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "a1")

    got, _ := p.GetPlan("p1")
    require.NotNil(t, got.Actions[0].Approved)
    assert.True(t, *got.Actions[0].Approved)
    assert.Nil(t, got.Actions[1].Approved)
}

func TestSlashPlan_RejectExitsPlanMode(t *testing.T) {
    c, p, mc := newPlanCommand(t)

```

```

    require.NoError(t, mc.TransitionTo(planmode.ModePlan))
    require.NoError(t, p.SubmitPlan(samplePlanForTest()))
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"reject"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "rejected")
    assert.Equal(t, planmode.ModeNormal, mc.GetMode())
}

func TestSlashPlan_StatusReportsMode(t *testing.T) {
    c, _, mc := newPlanCommand(t)
    require.NoError(t, mc.TransitionTo(planmode.ModePlan))
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "Plan")
}

func TestSlashPlan_UnknownSubcommandErrors(t *testing.T) {
    c, _, _ := newPlanCommand(t)
    _, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"bogus"}})
    require.Error(t, err)
}

```



Step 2: Run failing test

```
cd HelixCode && go test -count=1 -run "TestSlashPlan" ./internal/
commands/
```

Expected: FAIL with NewPlanCommand undefined.



Step 3: Write plan_command.go

Create helix_code/internal/commands/plan_command.go:

```

package commands

import (
    "context"
    "fmt"
    "strings"
    "text/tabwriter"

    "dev.helix.code/internal/workflow/planmode"
)

// PlanCommand implements the /plan slash command.

```

```

type PlanCommand struct {
    planner planmode.Planner
    mc      planmode.ModeController
}

func NewPlanCommand(planner planmode.Planner, mc
    planmode.ModeController) *PlanCommand {
    return &PlanCommand{planner: planner, mc: mc}
}

func (c *PlanCommand) Name() string      { return "plan" }
func (c *PlanCommand) Aliases() []string { return nil }
func (c *PlanCommand) Description() string { return "Inspect,
    approve, or reject the active plan." }
func (c *PlanCommand) Usage() string      { return "/plan [show|
    approve [<id>]|reject|status]" }

func (c *PlanCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error) {
    args := cc.Args
    sub := "show"
    if len(args) > 0 {
        sub = args[0]
    }
    switch sub {
    case "show":
        return c.show()
    case "approve":
        return c.approve(args[1:])
    case "reject":
        return c.reject()
    case "status":
        return c.status()
    default:
        return nil, fmt.Errorf("/plan: unknown subcommand %q
            (want show|approve|reject|status)", sub)
    }
}

func (c *PlanCommand) show() (*CommandResult, error) {
    plan := c.planner.ActivePlan()
    if plan == nil {
        return &CommandResult{Success: true, Output: "No active
            plan."}, nil
    }
    var sb strings.Builder
    fmt.Fprintf(&sb, "Plan: %s - %s\n", plan.ID, plan.Title)
    if plan.Description != "" {

```



```

        fmt.Fprintf(&sb, "Description: %s\n", plan.Description)
    }
    fmt.Fprintf(&sb, "Status: %s\n\n", plan.Status)
    tw := tabwriter.NewWriter(&sb, 0, 0, 2, ' ', 0)
    fmt.Fprintln(tw, "ID\tTOOL\tAPPROVED\tEXECUTED\tDESCRIPTION")
    for _, a := range plan.Actions {
        approved := "no"
        if a.Approved != nil && *a.Approved {
            approved = "yes"
        }
        executed := "no"
        if a.Executed {
            executed = "yes"
        }
        fmt.Fprintf(tw, "%s\t%s\t%s\t%s\t%s\n", a.ID,
            a.ToolName, approved, executed, a.Description)
    }
    tw.Flush()
    return &CommandResult{Success: true, Output: sb.String()},
        nil
}

func (c *PlanCommand) approve(rest []string) (*CommandResult,
    error) {
    plan := c.planner.ActivePlan()
    if plan == nil {
        return nil, fmt.Errorf("/plan approve: no active plan")
    }
    if len(rest) == 0 {
        if err := c.planner.ApprovePlan(plan.ID); err != nil {
            return nil, err
        }
        return &CommandResult{Success: true, Output:
            fmt.Sprintf("Plan %s approved (%d actions).", plan.ID,
                len(plan.Actions))}, nil
    }
    actionID := rest[0]
    if err := c.planner.ApproveAction(plan.ID, actionID); err !=
        nil {
        return nil, err
    }
    return &CommandResult{Success: true, Output:
        fmt.Sprintf("Approved action %s.", actionID)}, nil
}

func (c *PlanCommand) reject() (*CommandResult, error) {
    plan := c.planner.ActivePlan()
    if plan == nil {

```

```

        return nil, fmt.Errorf("/plan reject: no active plan")
    }
    if err := c.planner.RejectPlan(plan.ID); err != nil {
        return nil, err
    }
    if c.mc.GetMode() == planmode.ModePlan {
        _ = c.mc.TransitionTo(planmode.ModeNormal)
    }
    return &CommandResult{Success: true, Output:
        fmt.Sprintf("Plan %s rejected. Returning to normal
        mode.", plan.ID)}, nil
}

func (c *PlanCommand) status() (*CommandResult, error) {
    mode := c.mc.GetMode()
    plan := c.planner.ActivePlan()
    planSummary := "no active plan"
    if plan != nil {
        planSummary = fmt.Sprintf("plan=%s status=%s
        actions=%d", plan.ID, plan.Status, len(plan.Actions))
    }
    return &CommandResult{Success: true, Output:
        fmt.Sprintf("Mode: %s | %s", mode, planSummary)}, nil
}

```



Step 4: Run slash tests

```
cd HelixCode && go test -count=1 -race -run "TestSlashPlan" ./
internal/commands/
```

Expected: PASS (7/7).



Step 5: Add RegisterBuiltinCommandsWithPlanMode

Modify helix_code/internal/commands/builtin/register.go:

```

import (
    "dev.helix.code/internal/commands"
    "dev.helix.code/internal/workflow/planmode"
)

// RegisterBuiltinCommandsWithPlanMode registers all built-in
// commands plus the
// /plan slash command bound to the given Planner +
// ModeController.
func RegisterBuiltinCommandsWithPlanMode(registry
    *commands.Registry, planner planmode.Planner, mc
    planmode.ModeController) error {

```

```

    if err := RegisterBuiltinCommands(registry); err != nil {
        return err
    }
    return registry.Register(commands.NewPlanCommand(planner,
        mc))
}

```

Also add "plan" to GetBuiltinCommandNames() and add "plan" to the worktreeOnly/skip set in builtin_test.go's TestRegisterBuiltinCommands (mirror what F06 did for "mcp" and F07 did for "tasks").



Step 6: Add register test

Create helix_code/internal/commands/builtin/plan_register_test.go:

```

package builtin

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/commands"
    "dev.helix.code/internal/workflow/planmode"
)

func TestRegisterBuiltinCommandsWithPlanMode(t *testing.T) {
    registry := commands.NewRegistry()
    planner := planmode.NewDefaultPlanner()
    mc := planmode.NewModeController()
    require.NoError(t,
        RegisterBuiltinCommandsWithPlanMode(registry, planner,
            mc))

    cmd, ok := registry.Get("plan")
    require.True(t, ok)
    assert.Equal(t, "plan", cmd.Name())
}

```



Step 7: Run all command tests

```

cd HelixCode && go test -count=1 -race ./internal/commands/ ./
internal/commands/builtin/

```

Expected: PASS — TestSlashPlan_*, TestRegisterBuiltinCommandsWithPlanMode, plus existing tests for hooks/worktree/mcp/tasks/permissions still passing.



Step 8: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/commands/plan_command.go internal/
commands/builtin/register.go && echo "BLUFF FOUND" ||
echo "clean"
```

Expected: clean.



Step 9: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
helix_code/internal/commands/plan_command.go helix_code/
internal/commands/plan_command_test.go helix_code/
internal/commands/builtin/register.go helix_code/
internal/commands/builtin/plan_register_test.go
helix_code/internal/commands/builtin/builtin_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
feat(P1-F08-T06): add /plan slash command + builtin registration
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
EOF
)"
```

Task 7: cmd/cli/main.go startup wiring + integration test

Files: - Modify: `helix_code/cmd/cli/main.go` — construct `ModeController`, `Planner`, `ToolGate`; wire to registry + slash; register `Enter/ExitPlanMode` tools - Create: `helix_code/tests/integration/planmode_gating_test.go` (//go:build integration)



Step 1: Wire into main.go

Find the existing F07 wiring block in `main.go` (where `bgMgr := workflow.NewBackgroundManager(...)` is constructed). Add adjacent:

```
// F08: plan-mode gate
modeCtrl := planmode.NewModeController()
planner := planmode.NewDefaultPlanner()
// Optional: planner.SetHooksManager(hooksMgr) if a hooks manager
is wired.
gate := planmode.NewToolGate(modeCtrl, planner)
toolReg.SetPlanModeGate(gate)

// Register Enter/Exit plan-mode agent tools.
toolReg.Register(tools.NewEnterPlanModeTool(modeCtrl))
```

```

toolReg.Register(tools.NewExitPlanModeTool(modeCtrl))

// Register /plan slash command (direct register; matches F07
// pattern to avoid
// double-registering base commands through
// RegisterBuiltinCommandsWithPlanMode).
if err := cmdRegistry.Register(commands.NewPlanCommand(planner,
    modeCtrl)); err != nil {
    log.Printf("plan: register slash command failed: %v", err)
}

```

ADAPT variable names (toolReg, cmdRegistry, hooksMgr, etc.) to whatever main.go uses. Add the import "dev.helix.code/internal/workflow/planmode" if not present (it should already be imported by F07's workflow package usage).



Step 2: Verify cmd/cli builds

```
cd HelixCode && go build ./cmd/cli/...
```

Expected: clean.



Step 3: Write integration test

Create helix_code/tests/integration/planmode_gating_test.go:

```

//go:build integration

package integration

import (
    "context"
    "errors"
    "strings"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/tools"
    "dev.helix.code/internal/workflow/planmode"
)

func TestPlanMode_BlocksUnapprovedBash(t *testing.T) {
    if testing.Short() {
        t.Skip("SKIP-OK: integration only")
    }
}

```

```

reg, err :=
    tools.NewToolRegistry(tools.DefaultRegistryConfig())
require.NoError(t, err)
mc := planmode.NewModeController()
p := planmode.NewDefaultPlanner()
reg.SetPlanModeGate(planmode.NewToolGate(mc, p))

require.NoError(t, mc.TransitionTo(planmode.ModePlan))

ctx, cancel := context.WithTimeout(context.Background(),
    5*time.Second)
defer cancel()
_, err = reg.Execute(ctx, "shell", map[string]interface{}{
    {"command": "echo blocked"}}
require.Error(t, err)
assert.True(t, errors.Is(err, tools.ErrPlanModeGated))
}

func TestPlanMode_AllowsApprovedBash(t *testing.T) {
    if testing.Short() {
        t.Skip("SKIP-OK: integration only")
    }
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    require.NoError(t, err)
    mc := planmode.NewModeController()
    p := planmode.NewDefaultPlanner()
    reg.SetPlanModeGate(planmode.NewToolGate(mc, p))

    require.NoError(t, mc.TransitionTo(planmode.ModePlan))
    plan := &planmode.Plan{
        ID: "p1",
        Title: "T",
        Actions: []planmode.PlanAction{
            {ID: "a1", ToolName: "shell", Args:
                map[string]any{"command": "echo allowed"}, Description:
                "echo"},
        },
        Status: planmode.PlanPending,
    }
    require.NoError(t, p.SubmitPlan(plan))
    require.NoError(t, p.ApprovePlan("p1"))

    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
    defer cancel()
    res, err := reg.Execute(ctx, "shell", map[string]interface{}{
        {"command": "echo allowed"}}

```

```

require.NoError(t, err)
// Result is whatever the shell tool returns – just confirm
    "allowed" appears
rs := toString(res)
assert.Contains(t, rs, "allowed")
}

func TestPlanMode_ExitReturnsToNormal(t *testing.T) {
    if testing.Short() {
        t.Skip("SKIP-OK: integration only")
    }
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    require.NoError(t, err)
    mc := planmode.NewModeController()
    p := planmode.NewDefaultPlanner()
    reg.SetPlanModeGate(planmode.NewToolGate(mc, p))

    exitTool := tools.NewExitPlanModeTool(mc)
    reg.Register(exitTool)

    require.NoError(t, mc.TransitionTo(planmode.ModePlan))
    _, err = exitTool.Execute(context.Background(), nil)
    require.NoError(t, err)
    assert.Equal(t, planmode.ModeNormal, mc.GetMode())

    // Subsequent shell call now succeeds without an approved
    // plan.
    ctx, cancel := context.WithTimeout(context.Background(),
        5*time.Second)
    defer cancel()
    res, err := reg.Execute(ctx, "shell", map[string]interface{}{
        "command": "echo unblocked"})
    require.NoError(t, err)
    assert.Contains(t, toString(res), "unblocked")
}

func toString(v interface{}) string {
    if v == nil {
        return ""
    }
    if m, ok := v.(map[string]interface{}); ok {
        if s, ok := m["output"].(string); ok {
            return s
        }
    }
    return strings.TrimSpace(asJSON(v))
}

```

```

func asJSON(v interface{}) string {
    // Best-effort string extraction for assertion-friendly
    // output.
    switch x := v.(type) {
    case string:
        return x
    case map[string]interface{}:
        var sb strings.Builder
        for _, val := range x {
            fmt := strings.TrimSpace(asJSON(val))
            if fmt != "" {
                sb.WriteString(fmt + " ")
            }
        }
        return sb.String()
    default:
        return ""
    }
}

```



Step 4: Run integration tests

```

cd HelixCode && go test -count=1 -tags=integration -run
    "TestPlanMode_" ./tests/integration/...

```

Expected: PASS (3/3).



Step 5: Full unit + integration sweep

```

cd HelixCode
go test -count=1 -race ./internal/workflow/planmode/... ./
    internal/tools/ ./internal/commands/ ./internal/commands/
    builtin/ ./cmd/cli/...
go test -count=1 -tags=integration -run "TestPlanMode_" ./tests/
    integration/...

```

Pre-existing failures unrelated to F08 may persist; document but do not let them block.



Step 6: Cross-compile linux

```

cd HelixCode && go build ./cmd/cli/... ./internal/workflow/
    planmode/... ./internal/tools/ ./internal/commands/...

```

Expected: clean.



Step 7: Anti-bluff smoke (broad)

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/workflow/planmode/gating.go \
internal/tools/types_planmode.go internal/tools/plan_tools.go \
internal/commands/plan_command.go internal/commands/builtin/
register.go \
cmd/cli/main.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean (pre-existing bluff strings in OTHER files unrelated to F08 are out-of-scope).



Step 8: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
helix_code/cmd/cli/main.go helix_code/tests/integration/
planmode_gating_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
feat(P1-F08-T07): wire plan-mode gate into cmd/cli + integration
test (real Bash gating)
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 8: Challenge with runtime evidence + cross-compile check

Files: - Create: helix_code/tests/integration/cmd/p1f08_challenge/main.go — harness - Create: challenges/p1-f08-plan-mode/CHALLENGE.md (in Challenges submodule) - Create: challenges/p1-f08-plan-mode/run.sh - Modify: docs/improvements/06_phase_1_evidence.md — append T08 section



Step 1: Write challenge harness

Create helix_code/tests/integration/cmd/p1f08_challenge/main.go:

```
// p1f08_challenge runs the full plan-mode gating flow against a
real registry
// and a real shell tool. Runtime-evidence harness for the F08
Challenge.
package main

import (
    "context"
```

```

"errors"
"fmt"
"os"
"strings"

"dev.helix.code/internal/tools"
"dev.helix.code/internal/workflow/planmode"
)

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, "FAIL:", err)
        os.Exit(1)
    }
}

func run() error {
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    if err != nil {
        return fmt.Errorf("registry: %w", err)
    }
    mc := planmode.NewModeController()
    planner := planmode.NewDefaultPlanner()
    gate := planmode.NewToolGate(mc, planner)
    reg.SetPlanModeGate(gate)

    ctx := context.Background()

    fmt.Println("==> step 1: transition to plan mode")
    if err := mc.TransitionTo(planmode.ModePlan); err != nil {
        return fmt.Errorf("transition: %w", err)
    }
    if mc.GetMode() != planmode.ModePlan {
        return fmt.Errorf("expected ModePlan, got %v",
            mc.GetMode())
    }
    fmt.Println("    mode =", mc.GetMode())

    fmt.Println("==> step 2: try Bash without approved plan ->
        expect blocked")
    _, err = reg.Execute(ctx, "shell", map[string]interface{}{
        "command": "echo hi"})
    if err == nil {
        return fmt.Errorf("expected ErrPlanModeGated, got nil")
    }
    if !errors.Is(err, tools.ErrPlanModeGated) {

```

```

        return fmt.Errorf("expected ErrPlanModeGated, got: %v",
            err)
    }
    fmt.Println("    blocked correctly:", err)

    fmt.Println("==> step 3: submit + approve plan with Bash
        echo hi action")
    plan := &planmode.Plan{
        ID:      "p1f08",
        Title:    "F08 challenge",
        Actions: []planmode.PlanAction{
            {ID: "a1", ToolName: "shell", Args:
                map[string]any{"command": "echo hi"}, Description:
                "echo"},
        },
        Status: planmode.PlanPending,
    }
    if err := planner.SubmitPlan(plan); err != nil {
        return fmt.Errorf("submit: %w", err)
    }
    if err := planner.ApprovePlan("p1f08"); err != nil {
        return fmt.Errorf("approve: %w", err)
    }
    fmt.Println("    plan approved.")

    fmt.Println("==> step 4: run Bash echo hi -> expect success
        with 'hi' in output")
    res, err := reg.Execute(ctx, "shell", map[string]interface{}{
        "command": "echo hi"})
    if err != nil {
        return fmt.Errorf("expected success, got: %w", err)
    }
    out := outputString(res)
    fmt.Println("    output =", out)
    if !strings.Contains(out, "hi") {
        return fmt.Errorf("expected output to contain 'hi'; got
            %q", out)
    }

    fmt.Println("==> step 5: try Bash echo bye -> expect blocked
        (key-arg mismatch)")
    _, err = reg.Execute(ctx, "shell", map[string]interface{}{
        "command": "echo bye"})
    if err == nil {
        return fmt.Errorf("expected ErrPlanModeGated for
            unapproved command, got nil")
    }

```

```

if !errors.Is(err, tools.ErrPlanModeGated) {
    return fmt.Errorf("expected ErrPlanModeGated, got: %v",
        err)
}
fmt.Println("    blocked correctly (key-arg mismatch):", err)

fmt.Println("==> step 6: ExitPlanMode -> Bash echo bye now
    succeeds")
exitTool := tools.NewExitPlanModeTool(mc)
if _, err := exitTool.Execute(ctx, nil); err != nil {
    return fmt.Errorf("exit: %w", err)
}
res, err = reg.Execute(ctx, "shell", map[string]interface{}{
    "command": "echo bye"})
if err != nil {
    return fmt.Errorf("expected success in normal mode, got:
        %w", err)
}
out = outputString(res)
fmt.Println("    output =", out)
if !strings.Contains(out, "bye") {
    return fmt.Errorf("expected output to contain 'bye'; got
        %q", out)
}

fmt.Println("==> P1-F08 challenge harness PASS")
return nil
}

func outputString(v interface{}) string {
    if m, ok := v.(map[string]interface{}); ok {
        if s, ok := m["output"].(string); ok {
            return s
        }
    }
    if s, ok := v.(string); ok {
        return s
    }
    return ""
}

```

Verify it builds:

```
cd HelixCode && go build ./tests/integration/cmd/p1f08_challenge
```



Step 2: Write CHALLENGE.md

Create challenges/p1-f08-plan-mode/CHALLENGE.md:

Challenge: P1-F08 — Plan Mode

Purpose

Prove that HelixCode's plan-mode gating actually blocks unauthorised destructive tool calls, allows them once a matching plan action is approved, key-arg mismatches still block, and ExitPlanMode restores normal execution. Per Article XI §11.9, every PASS must carry positive runtime evidence.

Procedure

1. Build the F08 challenge harness.
2. Run the harness — it:
 - a. Transitions to ``ModePlan``.
 - b. Calls ``shell`` (echo hi) with NO approved plan — expects ``ErrPlanModeGated``.
 - c. Submits + approves a plan with action ``shell command=echo hi``.
 - d. Calls ``shell`` (echo hi) — expects success, captures "hi".
 - e. Calls ``shell`` (echo bye, different command) — expects ``ErrPlanModeGated``.
 - f. Calls ``ExitPlanMode``, then ``shell`` (echo bye) — expects success.
3. Anti-bluff smoke clean.
4. Cross-compile linux clean.

Pass criteria

- Harness exits 0 with ``==> P1-F08 challenge harness PASS`` final line.
- All five gating outcomes captured (blocked → allowed → blocked-on-mismatch → allowed-after-exit).
- Anti-bluff smoke clean.
- Cross-compile linux clean.



Step 3: Write run.sh

Create challenges/p1-f08-plan-mode/run.sh:

```
#!/usr/bin/env bash
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT="$(cd "$HERE/../../" && pwd)"
cd "$ROOT/HelixCode"
```

```

echo "==> build F08 challenge harness"
HARNESS_BIN="$(mktemp -d)/p1f08_challenge"
go build -o "$HARNESS_BIN" ./tests/integration/cmd/
    p1f08_challenge

echo "==> run harness"
"$HARNESS_BIN"

echo "==> anti-bluff smoke on F08-affected code"
if grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
    internal/workflow/planmode/gating.go \
    internal/tools/types_planmode.go internal/tools/
    plan_tools.go \
    internal/commands/plan_command.go; then
    echo "BLUFF FOUND" >&2
    exit 1
fi
echo "clean"

echo "==> cross-compile linux"
go build ./cmd/cli/... ./internal/workflow/planmode/... ./
    internal/tools/

echo "==> P1-F08 challenge PASS"

```



Step 4: Run challenge

```

chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
    challenges/p1-f08-plan-mode/run.sh
/run/media/milosvasic/DATA4TB/Projects/helix_code/challenges/p1-
    f08-plan-mode/run.sh 2>&1 | tee /tmp/p1f08-run.log

```

Expected: exit 0, final line ==> P1-F08 challenge PASS.



Step 5: Append runtime evidence

In docs/improvements/06_phase_1_evidence.md, find the F08 section header (added in T01). Append:

T08 – Challenge run

```

\\\\"bash
$ ./challenges/p1-f08-plan-mode/run.sh
[paste FULL contents of /tmp/p1f08-run.log here verbatim]
\\\\"

```

T08 – All commits in the F08 branch

```
\\`\\`\\`bash
$ git log --oneline | grep "P1-F08"
[paste actual output verbatim]
\\`\\`\\`
```

NO PARAPHRASING. NO FABRICATION.



Step 6: Commit (dual: submodule + meta-repo)

```
# Submodule first
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/Challenges
git add p1-f08-plan-mode/
git commit -m "$(cat <<'EOF'
feat(P1-F08-T08): challenge runner for plan-mode gating

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
EOF
)"

# Then meta-repo (submodule pointer + harness + evidence.md)
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add Challenges helix_code/tests/integration/cmd/
p1f08_challenge docs/improvements/06_phase_1_evidence.md
git commit -m "$(cat <<'EOF'
feat(P1-F08-T08): challenge with runtime evidence + cross-compile
check

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>
EOF
)"
```

Task 9: Feature 8 close-out + push to 4 remotes

Files: - Modify: docs/improvements/PROGRESS.md



Step 1: Update PROGRESS.md current focus

```
## Current focus
- **Active phase:** P1 – claude-code feature porting
- **Active feature:** (idle, awaiting next feature pick – F09
  candidate)
- **Active task:** –
```

- ****Last completed:**** P1-F08-T09 – Feature 8 (Plan Mode) close-out + push
- ****Owner:**** agent (Claude Opus 4.7)
- ****Started:**** 2026-05-04
- ****Last touched:**** 2026-05-05
- ****Blocked-on:**** none



Step 2: Tick all F08 task list items

Change every - [] to - [x] for items T01–T09 in the F08 task list block.



Step 3: Final verification sweep

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

Unit + race

```
go test -count=1 -race ./internal/workflow/planmode/... ./
    internal/tools/ ./internal/commands/ ./internal/commands/
    builtin/ ./cmd/cli/...
```

Integration

```
go test -count=1 -tags=integration -run "TestPlanMode_" ./tests/
    integration/...
```

Anti-bluff smoke (F08 files)

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/workflow/planmode/gating.go \
    internal/tools/types_planmode.go internal/tools/plan_tools.go \
    internal/commands/plan_command.go internal/commands/builtin/
    register.go \
    cmd/cli/main.go && echo "BLUFF FOUND" || echo "clean"
```

Cross-compile linux

```
go build ./cmd/cli/... ./internal/workflow/planmode/... ./
    internal/tools/
```

go vet

```
go vet ./internal/workflow/planmode/... ./internal/tools/ ./
    internal/commands/... ./cmd/cli/...
```

Final challenge re-run

```
chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
    challenges/p1-f08-plan-mode/run.sh
/run/media/milosvasic/DATA4TB/Projects/helix_code/challenges/p1-
    f08-plan-mode/run.sh
```

If any step fails, STOP and report. Do NOT push.



Step 4: Commit close-out

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
    improvements/PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
chore(P1-F08-T09): Feature 8 (Plan Mode) close-out

Co-Authored-By: Claude Opus 4.7 (1M context)
    <noreply@anthropic.com>
EOF
)"
```



Step 5: Push non-force to all 4 remotes

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    origin main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    github main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    gitlab main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
    upstream main
```

CRITICAL: NO --force. NO --force-with-lease. NO history rewrites. (CONST-043).

Self-review notes

1. **Spec coverage:** every spec section has a task — Planner extensions + hooks (T02), gating (T03), agent tools + sentinel (T04), registry wiring (T05), slash + builtin (T06), main.go wiring + integration (T07), challenge (T08), close-out (T09).
2. **TDD discipline:** every code task starts with a failing test, runs to confirm fail, implements minimally, reruns to pass, smoke + commit.
3. **Type consistency:** ToolGate, Planner, ModeController, Plan, PlanAction, PlanStatus, PlanApproved, PlanRejected, EnterPlanModeTool, ExitPlanModeTool, PlanCommand, RegisterBuiltinCommandsWithPlanMode, ErrPlanModeGated, OnPlanApproval, OnPlanReject — all spelled identically across tasks.
4. **Cross-platform:** no platform-specific code; all pure Go. Cross-compile check appears in T07 and T08.
5. **Anti-bluff:** every code-introducing task runs the FULL 4-term smoke pattern. T08 captures real runtime evidence (5 gating outcomes).
6. **No new external deps:** all needed packages already in go.mod since F07.
7. **Branch + push:** stays on main, non-force pushes to all four remotes (origin/github/gitlab/upstream) per programme convention.
8. **Pre-existing failures:** internal/tools/git, internal/tools/multiedit, internal/tools/shell race tests — out-of-scope from F02 onward; do not introduce new failures in any package T05/T06 touch.